

EF Core Project

Line-by-Line C# & Entity Framework Core Explanation

Study Guide

Models → Configuration → DbContext → Program.cs

This guide explains the code from the uploaded EF Core project in a beginner-friendly way, focusing on what each line means, why it exists, and how the pieces connect.

Core idea: C# classes describe the data and relationships; EF Core configuration describes database rules; DbContext connects those rules to SQL Server; Program.cs uses the resulting model.

1. Student.cs — The Student Model

```
using System;
using System.Collections.Generic;
using System.ComponentModel.DataAnnotations;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace EF_Rev.Models
{
    public class Student
    {
        public int StudentId { get; set; }
        public string StudentName { get; set; }
        public string StudentPhone { get; set; }
        public string StudentAddress { get; set; }
        public DateTime CreatedDate { get; private set; }

        public virtual ICollection<StudentCourses> StudentCourses
        {
            get;
            set;
        } = new HashSet<StudentCourses>();
    }
}
```

Using directives

using System; imports the System namespace so types such as **DateTime** can be written without the full name **System.DateTime**.

using System.Collections.Generic; gives access to collection types such as **ICollection<T>** and **HashSet<T>**.

using System.ComponentModel.DataAnnotations;, **System.Linq**, **System.Text**, and **System.Threading.Tasks** are not needed by the shown Student class; they are typical Visual Studio template imports.

Namespace

namespace EF_Rev.Models places the class inside the EF_Rev.Models namespace. A namespace groups related classes and helps prevent naming conflicts.

Class declaration

public class Student declares a public class named Student. A class is a blueprint from which Student objects can be created.

StudentId

```
public int StudentId { get; set; }
```

public means other code can access the property. **int** means whole number. **StudentId** is the property name. **get** reads the value and **set** changes it.

EF Core conventions recognize a property named **StudentId** in the Student entity as the primary key. The configuration later makes this explicit with **HasKey**.

StudentName, StudentPhone, StudentAddress

```
public string StudentName { get; set; }
public string StudentPhone { get; set; }
public string StudentAddress { get; set; }
```

These are ordinary string properties. They represent student data and are later configured with database rules such as required status and maximum length.

CreatedDate

```
public DateTime CreatedDate { get; private set; }
```

DateTime represents a date and time. **private set** means code outside the Student class can read CreatedDate but cannot directly assign a new value.

The database is configured later with **HasDefaultValueSql("GETDATE()")**, so SQL Server supplies the current date/time when a row is inserted.

The navigation property — the big line

```
public virtual ICollection<StudentCourses> StudentCourses
{
    get;
    set;
} = new HashSet<StudentCourses>();
```

public: accessible from other classes.

virtual: allows overriding in derived classes and supports EF Core lazy-loading proxies when lazy loading is explicitly enabled. **virtual alone does not enable lazy loading**.

ICollection<StudentCourses>: a collection contract whose elements are StudentCourses objects. It represents that one Student can be related to many StudentCourses rows.

StudentCourses: the property name and an EF Core navigation property. It does not represent a normal column in tblStudents; it represents a relationship.

{ get; set; }: the collection can be read and replaced.

= new HashSet<StudentCourses>(); when a new Student object is created, its collection starts empty instead of null. That makes **student.StudentCourses.Add(...)** safe without first assigning a collection.

Why HashSet? It is a collection designed around uniqueness. It also fits the idea that the same relationship should not be represented twice. The database itself additionally enforces uniqueness through the composite key.

2. Courses.cs — The Course Model

```
public class Courses
{
    public int CourseID { get; set; }
    public string CourseName { get; set; }

    public virtual ICollection<StudentCourses> StudentCourses
    { get; set; } = new HashSet<StudentCourses>();
}
```

CourseID identifies a course. **CourseName** stores its name.

The **StudentCourses** collection is the navigation property from the other side: one Course can be related to many StudentCourses rows.

Conceptually: **Student** → many **StudentCourses** and **Course** → many **StudentCourses**.

3. StudentCourses.cs — The Junction Model

```
public class StudentCourses
{
    public int StudentID { get; set; }
    public int CourseID { get; set; }

    public virtual Student Student { get; set; }
```

```

    public virtual Courses Course { get; set; }
}

```

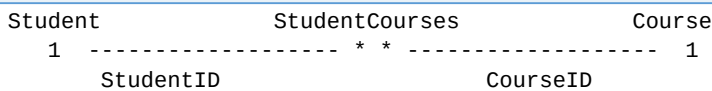
This class represents the middle/junction table for the many-to-many Student–Course relationship.

StudentID stores which student is involved. **CourseID** stores which course is involved.

Together, for example, (**StudentID = 1, CourseID = 5**) means Student 1 is enrolled in Course 5.

Student Student is singular because one StudentCourses row points to one Student. **Courses Course** is singular because one row points to one Course.

The overall structure is:



4. StudentConfiguration.cs — Database Rules

```

public class StudentConfiguration : IEntityTypeConfiguration<Student>
{
    public void Configure(EntityTypeBuilder<Student> builder)
    {
        builder.ToTable("tblStudents");
        builder.HasKey(s => s.StudentId);
        builder.Property(s => s.StudentName)
            .IsRequired()
            .HasMaxLength(150);

        builder.Property(s => s.CreatedDate)
            .HasDefaultValueSql("GETDATE()");
    }
}

```

IEntityTypeConfiguration<Student> means this class provides configuration rules specifically for Student.

Configure(EntityTypeBuilder<Student> builder) receives EF Core's builder object — the toolbox used to describe how Student maps to the database.

ToTable("tblStudents") maps the entity to the SQL table named tblStudents.

HasKey(s => s.StudentId) marks StudentId as the primary key. The lambda **s => s.StudentId** means: take a Student called s and return its StudentId.

Property(s => s.StudentName) selects the property being configured. **IsRequired()** means the database column cannot be NULL. **HasMaxLength(150)** limits the configured string length to 150.

HasDefaultValueSql("GETDATE()") tells SQL Server to use its GETDATE() function as the default when CreatedDate is not supplied.

5. CourseConfiguration.cs

```

builder.ToTable("tblCourses");
builder.HasKey(c => c.CourseID);
builder.Property(c => c.CourseName)
    .IsRequired()
    .HasMaxLength(150);

```

The pattern is the same: map the entity to **tblCourses**, make **CourseID** the primary key, and make **CourseName** required with a maximum configured length of 150.

6. StudentCoursesConfiguration.cs — Relationships

```

builder.HasKey(sc => new { sc.StudentID, sc.CourseID });

```

```
builder.HasOne(sc => sc.Student)
    .WithMany(s => s.StudentCourses)
    .HasForeignKey(sc => sc.StudentID);

builder.HasOne(sc => sc.Course)
    .WithMany(c => c.StudentCourses)
    .HasForeignKey(sc => sc.CourseID);
```

Composite primary key

`new { sc.StudentID, sc.CourseID }` creates an anonymous object containing both properties. EF Core uses the pair as a **composite primary key**.

Therefore the pair (**StudentID**, **CourseID**) must be unique. The same student cannot have the same course relationship inserted twice.

Student relationship

HasOne(sc => sc.Student) means one StudentCourses row has one Student.

WithMany(s => s.StudentCourses) means one Student can have many StudentCourses rows.

HasForeignKey(sc => sc.StudentID) says StudentID is the foreign key used for that relationship.

Course relationship

The Course block is the mirror image: one StudentCourses row has one Course; one Course has many StudentCourses rows; CourseID is the foreign key.

7. ApplicationDbContext.cs — The EF Core Bridge

```
public class ApplicationDbContext : DbContext
{
    protected override void OnConfiguring(
        DbContextOptionsBuilder optionsBuilder)
    {
        optionsBuilder.UseSqlServer(
            "Server=.;Database=StudentManagment;integrated security=true; Trust Server Certificate=true;");
    }

    public DbSet<Student> Students { get; set; }
    public DbSet<Courses> Courses { get; set; }
    public DbSet<StudentCourses> StudentCourses { get; set; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.ApplyConfiguration(new StudentConfiguration());
        modelBuilder.ApplyConfiguration(new CourseConfiguration());
        modelBuilder.ApplyConfiguration(new StudentCoursesConfiguration());
    }
}
```

: **DbContext** means ApplicationDbContext inherits from EF Core's DbContext. It receives EF Core functionality such as change tracking, querying, saving, and database communication.

OnConfiguring

protected override: OnConfiguring already exists in the base DbContext; override lets this class provide its own configuration.

UseSqlServer(...) selects SQL Server as the database provider. The connection string specifies the server, database name, authentication style, and trust setting.

DbSet properties

DbSet<Student> Students is an EF Core entry point for querying and managing Student entities. The same idea applies to Courses and StudentCourses.

```
dbContext.Students
dbContext.Courses
dbContext.StudentCourses
```

These are not simply ordinary in-memory lists. EF Core can translate operations such as LINQ queries into SQL.

OnModelCreating

ApplyConfiguration(new StudentConfiguration()) creates the configuration object and tells EF Core to apply its Configure rules. The same is done for CourseConfiguration and StudentCoursesConfiguration.

Without applying these configuration classes, the Fluent API rules in those files would not be applied through these calls.

8. Program.cs — Using the Model

```
var DbContext = new ApplicationDbContext();
```

new creates an object. **ApplicationDbContext()** creates the DbContext instance. **var** tells the compiler to infer the variable's type from the right-hand side. Here the inferred type is ApplicationDbContext.

Try / Catch

```
try
{
    // database-related operations
}
catch (Exception ex)
{
    Console.WriteLine(ex.Message);
}
```

The **try** block contains operations that might fail. The **catch** block handles an exception. **Exception ex** stores the error object in the variable ex, and **ex.Message** gives its human-readable message.

Reading students

```
var students = DbContext.Students.ToList();
```

DbContext.Students represents a query against the Student entity. **ToList()** executes/materializes the query and returns a List<Student>.

```
foreach (var student in students)
{
    Console.WriteLine(
        $"Student Name: {student.StudentName}, Address: {student.StudentAddress}, Phone: {student.StudentPhone}");
}
```

foreach processes each Student in the collection. **var student** is the current element. **\$"..."** is string interpolation, so expressions inside **{ }** are evaluated and inserted into the text.

The LINQ query

```
public static List<StudentCourseView>
    GetStudentCourseWithLINQ(ApplicationDbContext dbContext)
{
    var query =
        from std in dbContext.Students
        join sc in dbContext.StudentCourses
            on std.StudentId equals sc.StudentID
        join c in dbContext.Courses
            on sc.CourseID equals c.CourseID
        select new StudentCourseView
```

```

        {
            StudentName = std.StudentName,
            CourseName = c.CourseName
        };

    return query.ToList();
}

```

public static: the method is accessible and can be called without creating a Program object.

List<StudentCourseView> is the return type.

from std in dbContext.Students is similar to SQL FROM. Each Student is temporarily called **std**.

join sc in dbContext.StudentCourses on std.StudentId equals sc.StudentID connects students to junction rows where the IDs match. In SQL terms, this resembles an INNER JOIN with an ON condition.

join c in dbContext.Courses on sc.CourseID equals c.CourseID connects those junction rows to Courses.

select new StudentCourseView creates a new result object instead of returning entire Student and Course entities.

StudentName = std.StudentName and **CourseName = c.CourseName** copy only the two requested values into the result.

return query.ToList() materializes the query into a list and returns it. With EF Core's IQueryable, this is the point where the query is executed against the database.

Displaying StudentCourseView

```

public static void DisplayStudentCourses(
    List<StudentCourseView> studentCourses)
{
    foreach (var sc in studentCourses)
    {
        Console.WriteLine(
            $"Student Name: {sc.StudentName}, Course Name: {sc.CourseName}");
    }
}

```

The method receives a list of StudentCourseView objects and prints each one.

```

public class StudentCourseView
{
    public string StudentName { get; set; }
    public string CourseName { get; set; }
}

```

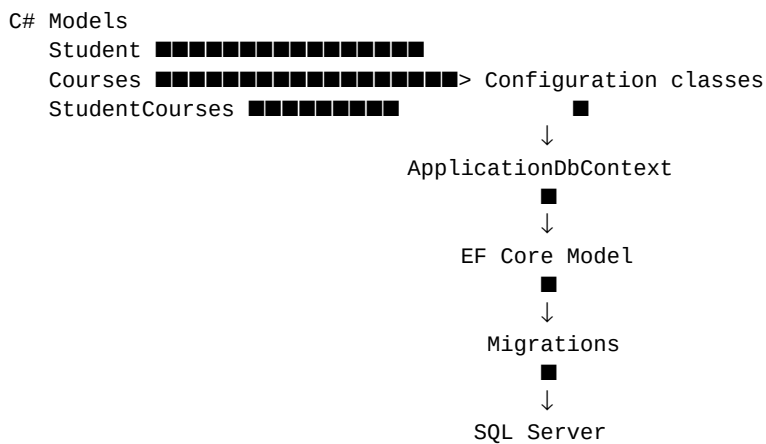
StudentCourseView is a small C# class used to shape the query result. It is not a database table just because it is a class; in this project it is not exposed as a DbSet.

9. C# Symbols You Keep Seeing

Symbol	Meaning
public	Accessible from other code.
private	Accessible only inside the class.
static	Belongs to the type rather than a particular object instance.
class	Defines a class/blueprint.
new	Creates an object or collection instance.
var	Compiler infers the variable's type.

Symbol	Meaning
{ }	Code block or property body, depending on context.
()	Method parameters or method calls.
< >	Generic type parameters, e.g. ICollection<StudentCourses>.
=>	Lambda expression operator; reads like 'maps/takes this and returns that'.
.	Accesses a member, e.g. student.StudentName.
;	Ends a C# statement.
:	Used for inheritance/interface implementation, e.g. : DbContext.

10. The Whole Project — How Everything Connects



Program.cs

```

■
■■■ creates ApplicationDbContext
■■■ queries DbSet<T>
■■■ writes LINQ
■■■ EF Core translates LINQ to SQL
■■■ displays the returned objects

```

Remember the three easily-confused concepts:

ICollection<StudentCourses> = a C# collection of related navigation objects.

DbSet<Student> = EF Core's entry point for querying/managing Student entities.

StudentCourses class = the entity representing the junction/enrollment table.

11. Study Checklist

- Understand what a class and an object are.
- Understand auto-properties: get and set.
- Understand private set and why CreatedDate uses it.
- Understand interface vs concrete class: ICollection vs HashSet.
- Understand navigation properties and why they are collections.
- Understand the Student–StudentCourses–Course many-to-many structure.
- Understand Fluent API and lambda expressions such as `s => s.StudentId`.
- Understand composite keys.

- Understand foreign keys and HasOne / WithMany / HasForeignKey.
- Understand DbContext and DbSet.
- Understand LINQ from / join / select.
- Understand why ToList() materializes the query.

This guide follows the explanation of the uploaded EF Core project and is intended as a study document. The most important goal is not memorizing syntax; it is being able to explain what each layer is responsible for and how data moves from C# objects to SQL Server and back.